

Week-5 Assignment: Spark Fundamentals & DataFrames

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Answer: The key limitations of traditional Hadoop MapReduce include:

- **Disk I/O Bottleneck:** MapReduce writes intermediate data to the physical hard disk after every map and reduce phase, making the process highly latent and slow.
- **Rigid Architecture:** MapReduce forces developers into a strict "Map" and then "Reduce" paradigm, which is not efficient for all types of data workflows.
- **Poor Performance for Iterative Processing:** Because of the constant disk read/write cycles, MapReduce is highly inefficient for iterative tasks like machine learning, whereas Spark resolves this by processing data in-memory.

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Answer: Machine learning algorithms are naturally iterative; they pass over the same dataset multiple times to optimize mathematical weights. In a disk-based system, every iteration requires fetching data from the hard drive. Spark solves this by caching the dataset directly in the RAM (In-Memory Computing). Subsequent iterations read the data directly from memory, which is exponentially faster than disk operations, drastically reducing the overall training and processing time.

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

Answer:

Python

```
clean_df = df.dropDuplicates(subset=["user_id", "transaction_date"])
```

Q4: Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

Answer:

Python

```
from pyspark.sql.functions import col, avg
```

```
avg_sales_df = df_sales.filter(col("region") == "West") \
```

```
.groupBy("product_category") \
    .agg(avg("sale_amount").alias("average_sale_amount"))
```

Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

Answer:

- **`.na.drop()`:** Removes entire rows from the DataFrame if they contain null values.
- **`.na.fill()`:** Retains the rows but replaces the null values with a specified default value.

Code Example:

Python

```
filled_df = df.na.fill("Unknown", subset=["status"])
```

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Answer:

Python

```
from pyspark.sql.functions import col
```

```
city_counts_df = df.groupBy("city") \
    .count() \
    .filter(col("count") > 100)
```

Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Answer: Immutability means a Spark DataFrame cannot be modified "in-place" once it is created. When performing data cleaning steps like dropping or renaming a column, Spark does not alter the original DataFrame. Instead, it generates a completely new DataFrame. Therefore, you must chain your transformations together or reassign the result to a variable (e.g., `df_clean = df.drop("bad_col")`) to capture the changes.

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Answer:

Python

```
from pyspark.sql.functions import col
```

```
filtered_df = df.filter(
    (col("age") >= 18) &
    (col("age") <= 30) &
    (col("subscription") == "Premium")
)
```

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Answer: It is crucial to handle nulls first to ensure data accuracy. While Spark's sum() and avg() functions automatically ignore null values, this can skew calculations like averages because the denominator (total count of valid records) decreases. Furthermore, if all values in an aggregation group are null, the result will also be null, which might cause errors in downstream processes that strictly expect numerical outputs.

Q10: Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

Answer:

Python

```
from pyspark.sql.functions import col
```

```
df_revised = df.withColumn("event_time", col("raw_timestamp").cast("timestamp")) \
    .drop("raw_timestamp")
```

Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Answer: A **Shuffle** occurs when Spark redistributes data across different partitions (and different worker nodes) so that related data ends up on the same node. For instance, during a groupBy(), all records with the same key must be grouped together, regardless of where they originally resided. It is considered a **Wide Transformation** because a single partition in the newly created DataFrame depends on data from multiple parent partitions. Shuffles are highly resource-intensive due to heavy network traffic and disk I/O.

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

Answer:

Python

```
from pyspark.sql.functions import col
```

```
cleaned_users_df = df.filter(
    col("email").isNotNull() & (col("username") != "")
)
```

Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Answer:

Python

```
from pyspark.sql.functions import min, max, mean
```

```
stats_df = df.agg(
    min("price").alias("min_price"),
    max("price").alias("max_price"),
    mean("price").alias("mean_price")
)
```

Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

Answer: When inferSchema=true encounters inconsistent date formats (e.g., some rows as "MM/DD/YYYY" and others as "YYYY-MM-DD"), it will fail to recognize them as valid Dates. To avoid failure, Spark defaults to reading that entire column as a plain StringType (text). The risk is that you lose native date-time querying capabilities and will be forced to write complex, manual string-parsing logic to convert them back into proper timestamps later in the pipeline.

Q15: Write a final processing pipeline that filters out duplicates, fills null prices with 0, and groups by store_id to calculate total revenue.

Answer:

Python

```
from pyspark.sql.functions import sum
```

```
final_pipeline_df = (
    df.dropDuplicates()
    .fillna({"price": 0})
    .groupBy("store_id")
```

```
.agg(sum("price").alias("total_revenue"))  
)
```